SQL
입문부터 실무까지

BlockSQL 샘플 데이터로 직접 따라 하는 단계별 가이드

1. 입문
조회 · 조건 · 정렬

2. 중급
집계 · 조인 · 서브쿼리

3. 고급
CTE · 윈도우 함수 · 분석

4. 실무
설계 · 인덱스 · 트랜잭션 · 보안

이 가이드 사용법

모든 예제는 BlockSQL의 **샘플 카페** 데이터베이스(SQLite)에서 실제로 실행한 결과를 함께 보여줍니다. 결과 표는 이 PDF를 만들 때 자동으로 실행해 넣은 것이라 그대로 따라 하면 같은 결과가 나옵니다.

1. BlockSQL을 열고 **설정** → **언어** → **한국어**를 고릅니다.
2. 연결 메뉴에서 **샘플 새로 만들기**를 누릅니다. (한국어 샘플 데이터가 만들어집니다.)
3. 오른쪽 위 **코드** 탭에 예제를 붙여 넣고 `⌘/Ctrl + Enter`로 실행합니다.

샘플 데이터 구조

테이블	열	설명
`customers`	id, name, city, email, grade, joined	고객 62명 (일부러 넣은 중복·빈 값 포함)
`products`	id, name, category, price	상품 15개, 가격은 원 단위 정수
`orders`	id, customer_id, product_id, qty, order_date, status	주문 500건 (2026년 1~9월)

`orders.customer_id`는 `customers.id`를, `orders.product_id`는 `products.id`를 가리킵니다(외래키).

> **표기 규칙**
> SQL 키워드는 대문자, 테이블·열 이름은 소문자로 씁니다. SQL은 대소문자를 구분하지 않지만, 이렇게 쓰면 읽기 쉽습니다. 예제는 SQLite 기준이며, PostgreSQL·MySQL과 다른 부분은 따로 표시했습니다.

목차

1부 · 입문

1. 데이터베이스와 테이블
2. SELECT와 FROM
3. WHERE로 거르기
4. 정렬과 개수 제한
5. 계산과 함수
6. NULL 다루기

2부 · 중급

7. 집계 함수
8. GROUP BY와 HAVING
9. 실행 순서
10. JOIN
11. 서브쿼리
12. 집합 연산

3부 · 고급

13. CTE (WITH)
14. 윈도우 함수
15. 실전 분석 패턴
16. 데이터 바꾸기와 트랜잭션

4부 · 실무

17. 스키마 설계
18. 인덱스와 실행 계획
19. 쿼리 성능
20. 동시성과 격리 수준
21. 보안
22. 운영과 데이터 품질
23. DB별 차이

부록

A. 치트시트
B. 연습문제 정답

1부 · 입문

데이터 꺼내 보기

원하는 행과 열만 골라서, 원하는 순서로 보는 법.

1. 데이터베이스와 테이블

데이터베이스는 **테이블**의 모음입니다. 테이블은 엑셀 시트와 비슷합니다. 한 줄(**행**, row)이 하나의 대상(고객 한 명, 주문 한 건)이고, 한 칸 위의 이름(**열**, column)이 그 대상의 속성입니다.

엑셀	SQL
파일	데이터베이스
시트	테이블
줄	행 (row, record)
열 제목	열 (column, field)

엑셀과 다른 점은 **열마다 자료형이 정해져 있다**는 것(숫자 열에는 숫자만)과, **행에 순서가 없다**는 것입니다. 순서가 필요하면 매번 `ORDER BY`로 요청해야 합니다.

2. SELECT와 FROM

SQL 문장의 기본 모양은 "**어떤 열을**(SELECT) **어디에서**(FROM) 가져와라"입니다.

```
SELECT name, city, grade
FROM customers
LIMIT 5;
```

name	city	grade
장도윤	인천	실버
장하준	부산	일반
한민준	*NULL*	실버
김도윤	수원	VIP
조지호	서울	일반

5행

`*`는 "모든 열"입니다. 처음 표를 볼 때는 편하지만, 실무에서는 필요한 열만 적는 것이 좋습니다(19장).

```sql
SELECT * FROM products LIMIT 3;
```

id	name	category	price
1	아메리카노	음료	4500
2	카페라테	음료	5000
3	녹차라테	음료	5500

3행

별칭 AS

결과 열 이름을 바꿀 때 `AS` 를 씁니다. 계산 결과에 이름을 붙일 때 특히 필요합니다.

```sql
SELECT name AS 상품명, price AS 가격, price * 1.1 AS 부가세포함
FROM products
LIMIT 3;
```

상품명	가격	부가세포함
아메리카노	4500	4950
카페라테	5000	5500
녹차라테	5500	6050

3행

> **BlockSQL에서는**
> 📋 *표에서* 블록이 `FROM` , *보여주기* 블록이 `SELECT` 의 열 하나입니다. *이름* 칸이 `AS` 입니다.

---

3. WHERE로 거르기

`WHERE` 는 조건에 맞는 행만 남깁니다.

```sql
SELECT name, city, grade
FROM customers
WHERE city = '서울';
```

name	city	grade
조지호	서울	일반
오민준	서울	VIP
박수아	서울	실버
김수아	서울	일반
윤현우	서울	골드

5행

비교 연산자

연산자	뜻	예
`=`	같다	`grade = 'VIP'`
`<>` , `!=`	다르다	`status <> '취소'`
`> >= < <=`	크기 비교	`price >= 10000`
`BETWEEN a AND b`	a 이상 b 이하	`price BETWEEN 3000 AND 5000`
`IN (…)`	목록 중 하나	`city IN ('서울', '부산')`
`LIKE`	패턴	`name LIKE '김%'` (김으로 시작)
`IS NULL`	비어 있다	`city IS NULL`

글자 값은 **작은따옴표**(`'서울'`)로 감쌉니다. 큰따옴표는 열 이름용입니다.

여러 조건: AND, OR, NOT

```sql
SELECT name, category, price
FROM products
WHERE category IN ('음료', '디저트')
  AND price BETWEEN 5000 AND 7000;
```

name	category	price
카페라테	음료	5000
녹차라테	음료	5500
레몬에이드	음료	5800
치즈케이크	디저트	6500
티라미수	디저트	7000

5행

> **주의: AND가 OR보다 먼저**
>
> `a OR b AND c` 는 `a OR (b AND c)` 로 계산됩니다. AND와 OR을 섞을 때는 항상 괄호를 치세요. BlockSQL 코드 모드가 이 실수를 경고합니다.

LIKE 패턴

`%` 는 "아무 글자 여러 개", `_` 는 "아무 글자 하나"입니다.

```sql
SELECT name FROM products WHERE name LIKE '%라테';
```

name
카페라테
녹차라테

2행

4. 정렬과 개수 제한

`ORDER BY`로 정렬합니다. 기본은 오름차순(`ASC`), 내림차순은 `DESC` 입니다. 여러 기준을 쉼표로 이어 쓸 수 있습니다.

```sql
SELECT name, category, price
FROM products
ORDER BY category, price DESC
LIMIT 6;
```

name	category	price
티라미수	디저트	7000
치즈케이크	디저트	6500
마카롱	디저트	2500
크루아상	베이커리	3800
베이글	베이커리	3500
텀블러	상품	25,000

6행

`LIMIT n OFFSET m`은 m개를 건너뛰고 n개를 가져옵니다(페이지 넘기기).

```sql
SELECT name, price FROM products ORDER BY price DESC LIMIT 3 OFFSET 3;
```

name	price
드립백 세트	12,000
샐러드	8500
샌드위치	7500

3행

중복 제거 DISTINCT

```sql
SELECT DISTINCT category FROM products ORDER BY category;
```

category
디저트
베이커리
상품
음료
푸드

5행

5. 계산과 함수

산술

```sql
SELECT id, qty, qty * 2 AS 두배, qty / 2 AS 정수나눗셈, qty * 1.0 / 2 AS 실수나눗셈
FROM orders
WHERE qty IS NOT NULL
LIMIT 3;
```

id	qty	두배	정수나눗셈	실수나눗셈
1	4	8	2	2
2	4	8	2	2
3	3	6	1	1.5

3행

> **주의: 정수 나눗셈**
>
> SQLite와 PostgreSQL에서 `3 / 2` 는 `1` 입니다. 소수가 필요하면 한쪽을 실수로 만드세요(`* 1.0`). MySQL은 `1.5` 를 돌려줍니다.

문자열

```sql
SELECT name,
       LENGTH(name)            AS 글자수,
       UPPER(email)            AS 대문자,
       name || ' (' || city || ')' AS 표시이름
FROM customers
WHERE city IS NOT NULL
LIMIT 3;
```

name	글자수	대문자	표시이름
장도윤	3	USER1@EXAMPLE.COM	장도윤 (인천)
장하준	3	USER2@EXAMPLE.COM	장하준 (부산)
김도윤	3	USER4@EXAMPLE.COM	김도윤 (수원)

3행

문자열 이어 붙이기는 SQLite·PostgreSQL에서 `||` , MySQL에서 `CONCAT(a, b)` 입니다.

날짜

SQLite에는 날짜 자료형이 없고 `'2026-03-15'` 같은 글자로 저장합니다. 이 형식(ISO 8601)이면 글자 비교만으로 날짜 순서가 맞습니다.

```sql
SELECT order_date,
       strftime('%Y', order_date) AS 연도,
       strftime('%m', order_date) AS 월,
       date(order_date, '+30 days') AS 삼십일후
FROM orders
LIMIT 3;
```

order_date	연도	월	삼십일후
2026-06-11	2026	06	2026-07-11
2026-01-20	2026	01	2026-02-19
2026-04-24	2026	04	2026-05-24

3행

PostgreSQL: `EXTRACT(MONTH FROM order_date)`, `order_date + INTERVAL '30 days'` · MySQL: `MONTH(order_date)`, `DATE_ADD(order_date, INTERVAL 30 DAY)`

조건에 따라 값 바꾸기: CASE

```sql
SELECT name, price,
       CASE
         WHEN price < 5000  THEN '저가'
         WHEN price < 10000 THEN '중가'
         ELSE '고가'
       END AS 가격대
FROM products
LIMIT 5;
```

name	price	가격대
아메리카노	4500	저가
카페라테	5000	중가
녹차라테	5500	중가
레몬에이드	5800	중가
크루아상	3800	저가

5행

> **BlockSQL에서는**
> *계산* 범주에 산술(+ − × ÷), 함수(연도·월·글자 수…), *만약 … 이면 … 아니면*(CASE) 블록이 있습니다. 나눗셈 블록은 자동으로 실수 나눗셈을 씁니다.

---

6. NULL 다루기

`NULL`은 "값이 없음/모름"입니다. 0도, 빈 글자 `''`도 아닙니다. 그래서 **`= NULL`은 항상 실패**하고, `IS NULL`을 써야 합니다.

```sql
SELECT
  (SELECT COUNT(*) FROM customers WHERE city = NULL)  AS 등호로_찾기,
  (SELECT COUNT(*) FROM customers WHERE city IS NULL) AS IS_NULL로_찾기;
```

등호로_찾기	IS_NULL로_찾기
0	5

1행

NULL이 섞인 계산은 결과도 NULL이 됩니다. 대신할 값을 정하려면 `COALESCE`를 씁니다.

```sql
SELECT name, COALESCE(city, '미입력') AS city
FROM customers
WHERE city IS NULL
LIMIT 3;
```

name	city
한민준	미입력
임서연	미입력
강수아	미입력

3행

> **세 가지 진리값**
>
> SQL의 조건은 참·거짓·**모름**(NULL) 세 가지입니다. WHERE는 "참"인 행만 남기므로, `city <> '서울'` 은 city가 NULL인 행도 버립니다. NULL까지 포함하려면 `city <> '서울' OR city IS NULL`.

연습문제 1

1. 가격이 10,000원 이상인 상품을 비싼 순으로 보여주세요.
2. 이메일이 빈 글자(`''`)인 고객의 이름을 보여주세요.
3. '취소'가 아닌 주문 중 수량(qty)이 4인 주문을 최근 날짜 순으로 5개만 보여주세요.

2부 · 중급

묶고, 잇고, 끼워 넣기

여러 행을 요약하고, 여러 테이블을 연결하는 법.

7. 집계 함수

집계 함수는 여러 행을 값 하나로 줄입니다.

```
SELECT COUNT(*)       AS 주문수,
       COUNT(qty)     AS 수량있는주문,
       SUM(qty)       AS 총수량,
       ROUND(AVG(qty), 2) AS 평균수량,
       MIN(order_date) AS 첫주문,
       MAX(order_date) AS 마지막주문
FROM orders;
```

주문수	수량있는주문	총수량	평균수량	첫주문	마지막주문
500	485	1181	2.44	2026-01-01	2026-09-28

1행

`COUNT(*)` 는 행 수, `COUNT(열)` 은 그 열이 NULL이 아닌 행 수입니다. 두 값이 다르면 빈 값이 있다는 뜻입니다. `COUNT(DISTINCT 열)` 은 종류 수입니다.

```
SELECT COUNT(DISTINCT customer_id) AS 주문한고객수 FROM orders;
```

주문한고객수
60

1행

8. GROUP BY와 HAVING

`GROUP BY` 는 같은 값끼리 묶어서 묶음마다 집계합니다.

```sql
SELECT status, COUNT(*) AS 건수
FROM orders
GROUP BY status
ORDER BY 건수 DESC;
```

status	건수
완료	325
배송중	90
취소	85

3행

여러 열로 묶을 수도 있습니다. 매출 = 수량 × 가격이므로 상품 테이블과 연결해야 합니다(10장). 우선 상품별 판매 수량만 봅니다.

```sql
SELECT product_id, SUM(qty) AS 판매수량
FROM orders
WHERE status = '완료'
GROUP BY product_id
ORDER BY 판매수량 DESC
LIMIT 5;
```

product_id	판매수량
10	78
4	73
11	58
2	57
8	56

5행

> **규칙: SELECT의 일반 열은 GROUP BY에도**
>
> 집계 함수로 감싸지 않은 열은 GROUP BY에 있어야 합니다. PostgreSQL은 오류를 내고, SQLite·MySQL(일부 설정)은 아무 값이나 골라 보여줘서 더 위험합니다.

HAVING: 묶은 뒤에 거르기

`WHERE`는 묶기 **전** 행을, `HAVING`은 묶은 **후** 그룹을 거릅니다. 집계 결과로 거르려면 HAVING을 씁니다.

```sql
SELECT customer_id, COUNT(*) AS 주문수
FROM orders
WHERE status <> '취소'
GROUP BY customer_id
HAVING COUNT(*) >= 12
ORDER BY 주문수 DESC;
```

customer_id	주문수
50	16
58	12
15	12

3행

> **BlockSQL에서는**
> *묶기* 블록이 GROUP BY, *묶은 뒤 조건* 블록이 HAVING입니다. 시트에서 열 이름을 누르고 *묶기*를 고르면 개수까지 자동으로 붙습니다.

---

9. 실행 순서

SQL은 적은 순서와 **실행되는 순서가 다릅니다**. 이걸 알면 대부분의 오류가 이해됩니다.

`FROM / JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`

- WHERE에서는 SELECT의 별칭을 쓸 수 없습니다(아직 만들어지지 않음). ORDER BY에서는 쓸 수 있습니다.
- WHERE에서 집계 함수를 쓸 수 없습니다(아직 묶지 않음) → HAVING을 씁니다.
- LIMIT은 맨 마지막이라, 정렬 없이 LIMIT을 쓰면 "아무 n개"가 나옵니다.

BlockSQL 코드 모드의 **설명** 탭은 쿼리를 이 순서대로 풀어서 보여줍니다.

---

10. JOIN

테이블을 나누어 저장하는 이유는 중복을 없애기 위해서입니다(17장). 보여줄 때는 다시 이어 붙여야 하는데, 이것이 `JOIN`입니다.

```sql
SELECT o.id, c.name AS 고객, p.name AS 상품, o.qty, p.price * o.qty AS 금액
FROM orders o
JOIN customers c ON c.id = o.customer_id
JOIN products  p ON p.id = o.product_id
ORDER BY o.id
LIMIT 5;
```

id	고객	상품	qty	금액
1	윤예준	샐러드	4	34,000
2	윤하은	레몬에이드	4	23,200
3	한하은	크루아상	3	11,400
4	조지호	원두 200g	2	30,000
5	정하준	녹차라테	1	5500

5행

`orders o` 처럼 테이블에 짧은 별칭을 붙이면 `o.qty` 로 어느 테이블의 열인지 밝힐 수 있습니다. 두 테이블에 같은 이름의 열(`id` , `name`)이 있으면 반드시 밝혀야 합니다.

JOIN 종류

종류	결과
`INNER JOIN` (= `JOIN`)	양쪽에 짝이 있는 행만
`LEFT JOIN`	왼쪽은 전부, 짝이 없으면 오른쪽 열은 NULL
`RIGHT JOIN`	오른쪽은 전부 (LEFT의 반대)
`FULL JOIN`	양쪽 전부 (MySQL 미지원)
`CROSS JOIN`	모든 조합 (행 수 = 왼쪽 × 오른쪽)

LEFT JOIN으로 "없는 것" 찾기

한 번도 주문하지 않은 고객: LEFT JOIN 후 오른쪽이 NULL인 행.

```sql
SELECT c.id, c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL;
```

id	name
61	한민준
62	강서윤

2행

카테고리별 매출

```sql
SELECT p.category, SUM(o.qty * p.price) AS 매출, COUNT(*) AS 주문수
FROM orders o
JOIN products p ON p.id = o.product_id
WHERE o.status = '완료'
GROUP BY p.category
ORDER BY 매출 DESC;
```

category	매출	주문수
상품	3,887,000	102
음료	1,135,400	81
디저트	744,000	61
푸드	726,000	37
베이커리	351,000	44

5행

> **주의: 행이 불어나는 JOIN (fan-out)**
>
> 1:N 관계를 잇면 1쪽 행이 N번 반복됩니다. 고객 테이블의 값을 합산한 뒤 주문과 JOIN하면 같은 값이 여러 번 더해집니다. **먼저 집계하고 나중에 JOIN**하거나, JOIN 전후로 `COUNT(*)` 를 비교해 확인하세요.

> **BlockSQL에서는**
>
> *연결* 블록을 끼우면 외래키(또는 `테이블_id` 이름)를 보고 `ON` 조건을 자동으로 채웁니다. *짝 있는 것만* = INNER, *왼쪽 다 유지* = LEFT.

---

11. 서브쿼리

쿼리 안에 넣은 쿼리입니다. 넣는 위치에 따라 역할이 다릅니다.

값 하나 (스칼라 서브쿼리)

```sql
SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

name	price
텀블러	25,000
머그컵	18,000
원두 200g	15,000
드립백 세트	12,000

4행

목록 (IN / EXISTS)

```sql
SELECT name, grade
FROM customers c
WHERE EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.id AND o.status = '취소'
)
AND grade = 'VIP';
```

name	grade
김도윤	VIP
오시우	VIP
임서연	VIP
장하준	VIP
오민준	VIP

5행

> **주의: NOT IN과 NULL**
>
> `NOT IN (서브쿼리)`의 결과에 NULL이 하나라도 있으면 전체가 빈 결과가 됩니다. "없는 것 찾기"에는 `NOT EXISTS`나 LEFT JOIN … IS NULL을 쓰세요.

표 (파생 테이블)

```sql
SELECT 등급, COUNT(*) AS 고객수
FROM (
  SELECT customer_id,
         CASE WHEN COUNT(*) >= 10 THEN '단골' ELSE '일반' END AS 등급
  FROM orders
  GROUP BY customer_id
) t
GROUP BY 등급;
```

등급	고객수
단골	20
일반	40

2행

---

12. 집합 연산

결과 두 개를 위아래로 합치거나 비교합니다. 열 개수와 자료형이 같아야 합니다.

연산	뜻
`UNION`	합치고 중복 제거
`UNION ALL`	합치기 (중복 유지, 더 빠름)
`INTERSECT`	양쪽에 다 있는 것
`EXCEPT`	왼쪽에만 있는 것 (MySQL 8.0.31+)

```sql
SELECT city FROM customers WHERE grade = 'VIP'
INTERSECT
SELECT city FROM customers WHERE grade = '골드'
ORDER BY city;
```

city
NULL
서울
수원
인천
제주

5행

연습문제 2

4. 도시별 고객 수를 많은 순으로 보여주세요. 도시가 비어 있는 고객은 '미입력'으로 묶어 주세요.
5. 상품별 완료 주문 매출(수량×가격)을 구하고, 매출이 300,000원 이상인 상품만 보여주세요.
6. 한 번도 팔리지 않은(완료 주문이 없는) 상품이 있는지 찾아보세요.

3부 · 고급

분석하는 SQL

긴 쿼리를 읽기 좋게 나누고, 순위·누적·전월 대비를 한 번에 계산하는 법.

13. CTE (WITH)

`WITH 이름 AS (쿼리)`로 중간 결과에 이름을 붙입니다. 서브쿼리를 겹겹이 넣는 대신 위에서 아래로 읽히게 만들어 줍니다. 실무 분석 쿼리는 대부분 CTE로 씁니다.

```
WITH sales AS (
  SELECT o.customer_id, SUM(o.qty * p.price) AS amount
  FROM orders o
  JOIN products p ON p.id = o.product_id
  WHERE o.status = '완료'
  GROUP BY o.customer_id
)
SELECT c.name, c.grade, s.amount
FROM sales s
JOIN customers c ON c.id = s.customer_id
ORDER BY s.amount DESC
LIMIT 5;
```

name	grade	amount
김민준	일반	340,600
강수아	일반	285,300
조지우	실버	247,300
한민준	실버	227,400
김수아	일반	220,700

5행

재귀 CTE: 빠진 날짜 채우기

자기 자신을 참조하는 CTE로 연속된 값을 만들 수 있습니다. 매출이 없는 달도 0으로 보여줘야 할 때 씁니다.

```sql
WITH RECURSIVE months(m) AS (
  SELECT '2026-01'
  UNION ALL
  SELECT strftime('%Y-%m', m || '-01', '+1 month') FROM months WHERE m < '2026-12'
),
monthly AS (
  SELECT substr(order_date, 1, 7) AS m, COUNT(*) AS n
  FROM orders GROUP BY 1
)
SELECT months.m AS 월, COALESCE(monthly.n, 0) AS 주문수
FROM months LEFT JOIN monthly ON monthly.m = months.m
ORDER BY 1;
```

월	주문수
2026-01	56
2026-02	59
2026-03	54
2026-04	47
2026-05	53
2026-06	60
2026-07	57
2026-08	47
2026-09	67
2026-10	0

12행 중 10행 표시

---

14. 윈도우 함수

GROUP BY는 행을 하나로 **줄이지만**, 윈도우 함수는 행을 그대로 두고 옆에 계산 결과를 **붙입니다**. 모양은 `함수() OVER (PARTITION BY … ORDER BY …)` 입니다.

- `PARTITION BY` : 어떤 묶음 안에서 계산할지 (생략하면 전체)
- `ORDER BY` : 묶음 안의 순서 (순위·누적에 필요)

순위: ROW_NUMBER, RANK, DENSE_RANK

```sql
SELECT name, category, price,
       ROW_NUMBER() OVER (ORDER BY price DESC) AS row_num,
       RANK()       OVER (ORDER BY price DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY price DESC) AS dense
FROM products
LIMIT 6;
```

name	category	price	row_num	rnk	dense
텀블러	상품	25,000	1	1	1
머그컵	상품	18,000	2	2	2
원두 200g	상품	15,000	3	3	3
드립백 세트	상품	12,000	4	4	4
샐러드	푸드	8500	5	5	5
샌드위치	푸드	7500	6	6	6

6행

같은 값일 때 ROW_NUMBER는 아무 순서로 1, 2를, RANK는 1, 1, 3을, DENSE_RANK는 1, 1, 2를 줍니다.

누적합과 전체 대비 비율

```sql
WITH m AS (
  SELECT substr(o.order_date, 1, 7) AS 월, SUM(o.qty * p.price) AS 매출
  FROM orders o JOIN products p ON p.id = o.product_id
  WHERE o.status = '완료'
  GROUP BY 1
)
SELECT 월, 매출,
       SUM(매출) OVER (ORDER BY 월) AS 누적매출,
       ROUND(100.0 * 매출 / SUM(매출) OVER (), 1) AS 비중
FROM m
ORDER BY 월;
```

월	매출	누적매출	비중
2026-01	642,800	642,800	9.4
2026-02	777,700	1,420,500	11.4
2026-03	1,019,300	2,439,800	14.9
2026-04	572,700	3,012,500	8.4
2026-05	1,244,800	4,257,300	18.2
2026-06	688,200	4,945,500	10.1
2026-07	639,300	5,584,800	9.3
2026-08	504,400	6,089,200	7.4
2026-09	754,200	6,843,400	11

9행

앞뒤 행 보기: LAG, LEAD (전월 대비)

```sql
WITH m AS (
  SELECT substr(o.order_date, 1, 7) AS 월, SUM(o.qty * p.price) AS 매출
  FROM orders o JOIN products p ON p.id = o.product_id
  WHERE o.status = '완료'
  GROUP BY 1
)
SELECT 월, 매출,
       LAG(매출) OVER (ORDER BY 월) AS 전월,
       ROUND(100.0 * (매출 - LAG(매출) OVER (ORDER BY 월)) / LAG(매출) OVER (ORDER BY 월), 1)
AS 증감률
FROM m
ORDER BY 월;
```

월	매출	전월	증감률
2026-01	642,800	*NULL*	*NULL*
2026-02	777,700	642,800	21
2026-03	1,019,300	777,700	31.1
2026-04	572,700	1,019,300	-43.8
2026-05	1,244,800	572,700	117.4
2026-06	688,200	1,244,800	-44.7
2026-07	639,300	688,200	-7.1
2026-08	504,400	639,300	-21.1
2026-09	754,200	504,400	49.5

9행

이동 평균 (프레임)

```
WITH d AS (
  SELECT order_date AS 날짜, COUNT(*) AS 주문수
  FROM orders WHERE order_date < '2026-01-11'
  GROUP BY 1
)
SELECT 날짜, 주문수,
       ROUND(AVG(주문수) OVER (ORDER BY 날짜 ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2)
AS 최근3일평균
FROM d;
```

날짜	주문수	최근3일평균
2026-01-01	1	1
2026-01-02	4	2.5
2026-01-03	4	3
2026-01-04	1	3
2026-01-05	2	2.33
2026-01-06	3	2
2026-01-08	2	2.33
2026-01-09	1	2
2026-01-10	1	1.33

9행

---

15. 실전 분석 패턴

현업에서 반복해서 나오는 질문과 정석 풀이입니다.

그룹별 상위 N개 (Top-N per group)

"카테고리마다 가장 많이 팔린 상품 1개": 윈도우 함수로 순위를 매기고 바깥에서 거릅니다.

```sql
WITH ranked AS (
  SELECT p.category, p.name, SUM(o.qty) AS 판매량,
         ROW_NUMBER() OVER (PARTITION BY p.category ORDER BY SUM(o.qty) DESC) AS rn
  FROM orders o JOIN products p ON p.id = o.product_id
  WHERE o.status = '완료'
  GROUP BY p.category, p.name
)
SELECT category, name, 판매량 FROM ranked WHERE rn = 1 ORDER BY 판매량 DESC;
```

category	name	판매량
상품	원두 200g	78
음료	레몬에이드	73
디저트	티라미수	56
푸드	샐러드	51
베이커리	크루아상	50

5행

중복 제거하고 최신 1건만

고객마다 마지막 주문 한 건. 같은 패턴으로 "중복 행 중 하나만 남기기"도 풉니다.

```sql
WITH last AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC, id
DESC) AS rn
  FROM orders
)
SELECT customer_id, order_date, status FROM last WHERE rn = 1 ORDER BY customer_id LIMIT
5;
```

customer_id	order_date	status
1	2026-08-21	완료
2	2026-08-27	배송중
3	2026-09-26	배송중
4	2026-09-10	완료
5	2026-08-15	완료

5행

피벗: 행을 열로

표준 SQL에는 PIVOT이 없어서 `CASE`를 집계 함수 안에 넣습니다.

```sql
SELECT substr(order_date, 1, 7) AS 월,
       SUM(CASE WHEN status = '완료'   THEN 1 ELSE 0 END) AS 완료,
       SUM(CASE WHEN status = '배송중' THEN 1 ELSE 0 END) AS 배송중,
       SUM(CASE WHEN status = '취소'   THEN 1 ELSE 0 END) AS 취소
FROM orders
GROUP BY 1
ORDER BY 1
LIMIT 4;
```

월	완료	배송중	취소
2026-01	31	9	16
2026-02	42	11	6
2026-03	41	8	5
2026-04	26	10	11

4행

PostgreSQL은 `COUNT(*) FILTER (WHERE status = '완료')` 로 더 짧게 쓸 수 있습니다.

코호트 (가입 월별 재구매)

```sql
WITH first_order AS (
  SELECT customer_id, MIN(substr(order_date, 1, 7)) AS cohort
  FROM orders GROUP BY customer_id
),
activity AS (
  SELECT DISTINCT o.customer_id, f.cohort, substr(o.order_date, 1, 7) AS m
  FROM orders o JOIN first_order f ON f.customer_id = o.customer_id
)
SELECT cohort AS 첫주문월,
       COUNT(DISTINCT customer_id) AS 고객수,
       COUNT(DISTINCT CASE WHEN m > cohort THEN customer_id END) AS 이후재구매
FROM activity
GROUP BY cohort
ORDER BY cohort;
```

첫주문월	고객수	이후재구매
2026-01	39	39
2026-02	13	13
2026-03	5	5
2026-04	3	3

4행

중복 데이터 찾기

```sql
SELECT name, email, COUNT(*) AS 개수
FROM customers
GROUP BY name, email
HAVING COUNT(*) > 1;
```

name	email	개수
강서윤	user17@example.com	2
한민준	user3@example.com	2

2행

---

16. 데이터 바꾸기와 트랜잭션

아래 예제는 이 가이드를 만들 때 트랜잭션 안에서 실행하고 되돌렸습니다. BlockSQL에서는 실행 전에 몇 행이 바뀌는지 먼저 보여줍니다.

INSERT · UPDATE · DELETE

```sql
INSERT INTO products (name, category, price) VALUES ('바닐라라테', '음료', 5500);
UPDATE products SET price = price + 500 WHERE category = '음료';
DELETE FROM orders WHERE status = '취소';
SELECT name, price FROM products WHERE category = '음료';
```

name	price
아메리카노	5000
카페라테	5500
녹차라테	6000
레몬에이드	6300
바닐라라테	6000

5행 · 앞 문장들로 91행 변경 후 되돌림

> **WHERE 없는 UPDATE/DELETE**
>
> 테이블의 모든 행이 바뀌거나 지워집니다. 실행 전에 같은 WHERE로 `SELECT COUNT(*)` 를 먼저 돌려 보는 습관을 들이세요.

UPSERT: 있으면 수정, 없으면 추가

```sql
-- SQLite / PostgreSQL
INSERT INTO products (id, name, category, price) VALUES (1, '아메리카노', '음료', 4700)
ON CONFLICT (id) DO UPDATE SET price = excluded.price;

-- MySQL
INSERT INTO products (id, name, category, price) VALUES (1, '아메리카노', '음료', 4700)
ON DUPLICATE KEY UPDATE price = VALUES(price);
```

트랜잭션

여러 문장을 **전부 성공하거나 전부 없던 일**로 묶습니다. 계좌 이체처럼 중간에 멈추면 안 되는 작업에 필수입니다.

```sql
BEGIN;
UPDATE accounts SET balance = balance - 10000 WHERE id = 1;
UPDATE accounts SET balance = balance + 10000 WHERE id = 2;
COMMIT;   -- 문제가 있으면 ROLLBACK;
```

연습문제 3

7. 등급(grade)별로 완료 매출 1위 고객을 구하세요.
8. 월별 주문 수와 전월 대비 증감(건수)을 구하세요.
9. 상품별 매출과 함께, 전체 매출 중 비중(%)과 매출 순위를 한 번에 보여주세요.

4부 · 실무

운영 환경의 SQL

수백만 행, 여러 사용자, 실수하면 안 되는 데이터를 다루는 법.

17. 스키마 설계

정규화

같은 사실은 한 곳에만 저장합니다. 주문 테이블에 고객 이름을 복사해 두면, 이름이 바뀔 때 모든 주문을 고쳐야 하고 하나라도 빠뜨리면 데이터가 어긋납니다.

단계	규칙	어긴 예
1NF	한 칸에 값 하나	`tags = '커피,디저트'`
2NF	키 전체에 의존	(주문id, 상품id) 키인데 상품명 저장
3NF	키가 아닌 열에 의존 금지	우편번호와 함께 시/도를 반복 저장

분석용 데이터 웨어하우스에서는 조회 속도를 위해 일부러 합쳐 두기도 합니다(비정규화, 스타 스키마). 원본(운영 DB)은 정규화, 분석용 사본은 비정규화가 일반적입니다.

제약 조건으로 잘못된 데이터 막기

```sql
CREATE TABLE orders (
  id          BIGINT PRIMARY KEY,
  customer_id BIGINT NOT NULL REFERENCES customers(id),
  product_id  BIGINT NOT NULL REFERENCES products(id),
  qty         INTEGER NOT NULL CHECK (qty > 0),
  amount      NUMERIC(12, 2) NOT NULL,          -- 돈은 실수형(FLOAT) 금지
  status      TEXT NOT NULL CHECK (status IN ('paid', 'shipped', 'cancelled')),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(), -- 시간대 포함
  UNIQUE (customer_id, product_id, created_at)
);
```

- **돈**: `FLOAT`은 0.1 + 0.2 ≠ 0.3 같은 오차가 납니다. `NUMERIC/DECIMAL` 또는 최소 단위 정수(원, 센트)로 저장합니다.
- **시간**: 시간대를 포함한 타입(PostgreSQL `TIMESTAMPTZ`)으로 UTC 기준 저장, 보여줄 때 변환합니다.
- **키**: 의미 없는 대리키(`id`)를 기본키로, 업무상 고유한 값(이메일, 사업자번호)에는 `UNIQUE`를 겁니다.
- **NULL 허용은 예외로**: 기본은 `NOT NULL`, 정말 "모름"이 가능한 열만 NULL 허용.

18. 인덱스와 실행 계획

인덱스는 책 뒤의 찾아보기입니다. 없으면 DB는 모든 행을 읽어야 하고(전체 스캔), 있으면 정렬된 트리(B-tree)에서 바로 찾아갑니다.

실행 계획 보기

```
EXPLAIN QUERY PLAN
SELECT * FROM orders WHERE customer_id = 7;
```

detail
SCAN orders

1행

`SCAN` 은 전체를 다 읽는다는 뜻입니다. 인덱스를 만들고 다시 봅니다.

```
CREATE INDEX idx_orders_customer ON orders (customer_id);
EXPLAIN QUERY PLAN
SELECT * FROM orders WHERE customer_id = 7;
```

detail
SEARCH orders USING INDEX idx_orders_customer (customer_id=?)

1행 · 앞 문장들로 85행 변경 후 되돌림

`SEARCH … USING INDEX` 로 바뀌었습니다. PostgreSQL은 `EXPLAIN ANALYZE`, MySQL은 `EXPLAIN` (8.0.18+는 `EXPLAIN ANALYZE`)으로 실제 소요 시간까지 볼 수 있습니다.

복합 인덱스는 순서가 중요

`(customer_id, order_date)` 인덱스는 전화번호부처럼 customer_id로 먼저 정렬되어 있습니다.

- `WHERE customer_id = 7` ✅ · `WHERE customer_id = 7 AND order_date >= '2026-05-01'` ✅
- `WHERE order_date >= '2026-05-01'` ❌ (앞 열 없이 뒤 열만으로는 못 씀)

규칙: **같다(=) 조건 열을 앞에, 범위 조건 열을 뒤에**.

인덱스를 못 쓰게 만드는 습관

나쁜 예	좋은 예
`WHERE strftime('%Y', order_date) = '2026'`	`WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01'`
`WHERE name LIKE '%라테'`	`WHERE name LIKE '라테%'` 또는 전문 검색 인덱스
`WHERE phone = 01012345678` (글자 열에 숫자)	`WHERE phone = '01012345678'`
`WHERE a = 1 OR b = 2`	두 쿼리를 `UNION ALL`

> **인덱스의 비용**
>
> 인덱스는 읽기를 빠르게 하는 대신 INSERT·UPDATE를 느리게 하고 공간을 씁니다. 실제로 자주 쓰는 WHERE·JOIN·ORDER BY 열에만 만드세요.

19. 쿼리 성능

- **필요한 열만**: `SELECT *`는 네트워크·메모리를 낭비하고 커버링 인덱스를 못 쓰게 합니다.
- **N+1 피하기**: 고객 100명을 가져온 뒤 고객마다 주문을 따로 조회(101번)하지 말고 JOIN이나 `WHERE customer_id IN (…)` 한 번으로.
- **먼저 거르고 나중에 잇기**: 큰 테이블끼리 JOIN하기 전에 CTE에서 기간·상태로 줄입니다.
- **일괄 처리**: 1만 행을 넣을 때 INSERT 1만 번이 아니라 여러 행 VALUES + 하나의 트랜잭션.

페이지네이션: OFFSET 대신 키셋

`OFFSET 100000`은 앞의 10만 행을 읽고 버립니다. 마지막으로 본 키 다음부터 가져오면 페이지가 깊어져도 빠릅니다.

```sql
-- 다음 페이지: 직전 페이지 마지막 id가 120이었다면
SELECT id, order_date, status
FROM orders
WHERE id > 120
ORDER BY id
LIMIT 5;
```

id	order_date	status
121	2026-09-08	완료
122	2026-05-05	완료
123	2026-05-08	완료
124	2026-05-03	완료
125	2026-09-12	완료

5행

20. 동시성과 격리 수준

여러 사용자가 동시에 같은 데이터를 읽고 쓰면 이상 현상이 생깁니다. 트랜잭션의 **ACID** 성질(원자성·일관성·격리성·지속성) 중 격리성의 강도를 정하는 것이 격리 수준입니다.

격리 수준	더티 리드	반복 불가 읽기	팬텀	기본값
READ UNCOMMITTED	가능	가능	가능	
READ COMMITTED	방지	가능	가능	PostgreSQL
REPEATABLE READ	방지	방지	가능*	MySQL InnoDB
SERIALIZABLE	방지	방지	방지	SQLite

* PostgreSQL·InnoDB의 REPEATABLE READ는 구현상 팬텀도 대부분 막습니다.

잃어버린 갱신 막기

두 사람이 재고 10을 동시에 읽고 각자 1을 빼서 9를 저장하면 하나가 사라집니다. 해결책 세 가지:

```sql
-- 1) 계산을 DB에 맡기기 (가장 간단)
UPDATE products SET stock = stock - 1 WHERE id = 3 AND stock > 0;

-- 2) 비관적 잠금: 읽을 때 잠그기
BEGIN;
SELECT stock FROM products WHERE id = 3 FOR UPDATE;
UPDATE products SET stock = 9 WHERE id = 3;
COMMIT;

-- 3) 낙관적 잠금: 버전 열로 충돌 감지 (바뀐 행이 0이면 다시 시도)
UPDATE products SET stock = 9, version = version + 1
WHERE id = 3 AND version = 41;
```

> **데드락**
> 두 트랜잭션이 서로가 잡은 행을 기다리면 DB가 한쪽을 강제로 취소합니다. 항상 **같은 순서**(예: id 오름차순)로 행을 잠그고, 트랜잭션은 짧게 유지하고, 취소되면 재시도하도록 코드를 짭니다.

---

21. 보안

SQL 인젝션

사용자 입력을 SQL 문자열에 그대로 붙이면, 입력에 SQL을 넣어 DB를 조작할 수 있습니다.

```sql
-- 위험: 입력이  ' OR '1'='1  이면 모든 사용자가 조회됨
"SELECT * FROM users WHERE email = '" + input + "'"

-- 안전: 값은 항상 파라미터로 따로 전달
SELECT * FROM users WHERE email = ?        -- SQLite, MySQL
SELECT * FROM users WHERE email = $1       -- PostgreSQL
```

파라미터는 값으로만 취급되므로 SQL로 해석되지 않습니다. 테이블·열 이름처럼 파라미터로 못 넘기는 부분은 허용 목록(allow-list)으로 검사합니다.

최소 권한

```sql
CREATE ROLE report_reader LOGIN PASSWORD '…';
GRANT SELECT ON orders, products TO report_reader;   -- 읽기만
REVOKE ALL ON customers FROM report_reader;          -- 개인정보 차단
```

- 애플리케이션 계정은 필요한 테이블에 필요한 권한만. DDL(DROP 등) 권한은 마이그레이션 계정에만.
- 개인정보(이메일·전화번호)는 분석용 뷰에서 가리거나(마스킹) 해시합니다.
- 비밀번호는 DB에 평문 저장 금지 — bcrypt/argon2 해시로 저장합니다.

22. 운영과 데이터 품질

마이그레이션

운영 DB의 구조 변경은 손으로 하지 않고 **버전 관리되는 마이그레이션 파일**로 합니다(Flyway, Liquibase, Alembic, Prisma 등). 원칙:

- 한 번 적용한 마이그레이션은 고치지 않고 새 파일을 추가합니다.
- 큰 테이블에 열 추가·인덱스 생성은 잠금을 피하는 방식으로(PostgreSQL `CREATE INDEX CONCURRENTLY`).
- 열 이름 변경·삭제는 "추가 → 양쪽 쓰기 → 옮기기 → 옛 열 제거" 여러 단계로 나눠 배포 중에도 앱이 깨지지 않게 합니다.

뷰

자주 쓰는 복잡한 쿼리를 이름 붙여 저장합니다. 권한을 뷰 단위로 주면 원본 테이블을 숨길 수 있습니다.

```
CREATE VIEW v_sales AS
SELECT o.id, o.order_date, c.city, p.category, o.qty * p.price AS amount
FROM orders o
JOIN customers c ON c.id = o.customer_id
JOIN products p ON p.id = o.product_id
WHERE o.status = '완료';

SELECT city, SUM(amount) AS 매출 FROM v_sales GROUP BY city ORDER BY 매출 DESC LIMIT 3;
```

city	매출
광주	1,220,700
인천	851,400
제주	846,500

3행 · 앞 문장들로 85행 변경 후 되돌림

계산이 무거우면 결과를 저장해 두는 **머티리얼라이즈드 뷰**(PostgreSQL)를 쓰고 주기적으로 `REFRESH` 합니다.

데이터 품질 검사

BlockSQL의 **건강검진**이 실행하는 검사를 직접 쓰면 이렇습니다. 데이터 파이프라인에서는 이런 검사를 매일 자동으로 돌리고, 기준을 넘으면 알림을 보냅니다(dbt test, Great Expectations).

```
SELECT COUNT(*)                                    AS 전체,
       COUNT(*) - COUNT(city)                      AS city_빈값,
       SUM(CASE WHEN TRIM(email) = '' THEN 1 ELSE 0 END) AS email_빈글자,
       COUNT(DISTINCT email)                       AS email_종류,
       (SELECT COUNT(*) FROM orders o
        WHERE NOT EXISTS (SELECT 1 FROM customers c WHERE c.id = o.customer_id)) AS 고아주
문
FROM customers;
```

전체	city_빈값	email_빈글자	email_종류	고아주문
62	5	5	56	0

1행

코드 리뷰 체크리스트

- UPDATE/DELETE에 WHERE가 있는가? 영향 행 수를 먼저 확인했는가?
- JOIN 후 행 수가 예상과 같은가? (fan-out)
- NULL이 섞여도 결과가 맞는가? (`NOT IN`, `<>`, 평균)
- GROUP BY에 빠진 열은 없는가?
- WHERE 열에 함수를 씌워 인덱스를 못 쓰게 하지 않았는가?
- 사용자 입력은 모두 파라미터로 넘겼는가?
- 큰 결과에 LIMIT이나 기간 조건이 있는가?

---

23. DB별 차이

기능	SQLite	PostgreSQL	MySQL
개수 제한	`LIMIT n`	`LIMIT n` / `FETCH FIRST n ROWS ONLY`	`LIMIT n`
문자열 잇기	`a || b`	`a || b`	`CONCAT(a, b)`
연도 뽑기	`strftime('%Y', d)`	`EXTRACT(YEAR FROM d)`	`YEAR(d)`
오늘	`date('now')`	`CURRENT_DATE`	`CURDATE()`
자동 증가 키	`INTEGER PRIMARY KEY`	`GENERATED ALWAYS AS IDENTITY`	`AUTO_INCREMENT`
UPSERT	`ON CONFLICT … DO UPDATE`	`ON CONFLICT … DO UPDATE`	`ON DUPLICATE KEY UPDATE`
대소문자 무시 검색	`LIKE` (ASCII만)	`ILIKE`	`LIKE` (콜레이션 따라)
참/거짓	0 / 1	`BOOLEAN`	`TINYINT(1)`
식별자 따옴표	`"name"`	`"name"`	`` `name` ``
FULL JOIN	✅ (3.39+)	✅	❌
자료형 검사	느슨함 (STRICT 테이블 선택)	엄격	모드에 따라

BlockSQL의 블록은 연결한 DB 종류에 맞춰 이 차이를 자동으로 반영한 SQL을 만듭니다.

부록

치트시트와 정답

A. 치트시트

```
SELECT   [DISTINCT] 열, 집계(열) AS 별칭      -- 5. 보여줄 것
FROM     표 별칭                               -- 1. 어디서
[LEFT] JOIN 다른표 ON 연결조건                 -- 1. 잇기
WHERE    행 조건                               -- 2. 행 거르기
GROUP BY 묶을 열                               -- 3. 묶기
HAVING   그룹 조건                             -- 4. 그룹 거르기
ORDER BY 열 [DESC]                             -- 6. 정렬
LIMIT    n [OFFSET m];                         -- 7. 자르기
```

하고 싶은 것	쓰는 것
비어 있는 값 찾기	`IS NULL`
빈 값 대신 기본값	`COALESCE(a, 0)`
조건별 값	`CASE WHEN … END`
그룹별 개수	`GROUP BY` + `COUNT(*)`
집계로 거르기	`HAVING`
없는 것 찾기	`LEFT JOIN … IS NULL`, `NOT EXISTS`

하고 싶은 것	쓰는 것
순위	`ROW_NUMBER() OVER (…)`
그룹별 상위 N	순위 + 바깥 `WHERE rn <= N`
누적합	`SUM(x) OVER (ORDER BY …)`
전월 대비	`LAG(x) OVER (ORDER BY 월)`
행 → 열	`SUM(CASE WHEN …)`
긴 쿼리 정리	`WITH 이름 AS (…)`

B. 연습문제 정답

정답은 하나가 아닙니다. 같은 결과가 나오면 맞습니다.

1.

```sql
SELECT name, price FROM products WHERE price >= 10000 ORDER BY price DESC;
```

name	price
텀블러	25,000
머그컵	18,000
원두 200g	15,000
드립백 세트	12,000

4행

2.

```sql
SELECT name FROM customers WHERE email = '';
```

name
조지호
박현우
이수아
최수아
윤하은

5행

3.

```sql
SELECT id, customer_id, order_date, qty
FROM orders
WHERE status <> '취소' AND qty = 4
ORDER BY order_date DESC
LIMIT 5;
```

id	customer_id	order_date	qty
41	34	2026-09-28	4
195	46	2026-09-28	4
17	33	2026-09-19	4
394	50	2026-09-18	4
462	50	2026-09-16	4

5행

4.

```sql
SELECT COALESCE(city, '미입력') AS city, COUNT(*) AS 고객수
FROM customers
GROUP BY COALESCE(city, '미입력')
ORDER BY 고객수 DESC;
```

city	고객수
인천	11
광주	9
수원	8
대구	8
부산	7
제주	5
서울	5
미입력	5
대전	4

9행

5.

```sql
SELECT p.name, SUM(o.qty * p.price) AS 매출
FROM orders o JOIN products p ON p.id = o.product_id
WHERE o.status = '완료'
GROUP BY p.name
HAVING SUM(o.qty * p.price) >= 300000
ORDER BY 매출 DESC;
```

name	매출
텀블러	1,175,000
원두 200g	1,170,000
머그컵	846,000
드립백 세트	696,000
샐러드	433,500
레몬에이드	423,400
티라미수	392,000

7행

6.

```sql
SELECT p.id, p.name
FROM products p
WHERE NOT EXISTS (
  SELECT 1 FROM orders o WHERE o.product_id = p.id AND o.status = '완료'
);
```

id	name

0행

결과가 비어 있으면 모든 상품이 한 번 이상 팔린 것입니다.

7.

```sql
WITH s AS (
  SELECT c.grade, c.name, SUM(o.qty * p.price) AS amount,
         ROW_NUMBER() OVER (PARTITION BY c.grade ORDER BY SUM(o.qty * p.price) DESC) AS
rn
  FROM orders o
  JOIN customers c ON c.id = o.customer_id
  JOIN products p ON p.id = o.product_id
  WHERE o.status = '완료'
  GROUP BY c.id, c.grade, c.name
)
SELECT grade, name, amount FROM s WHERE rn = 1 ORDER BY amount DESC;
```

grade	name	amount
일반	김민준	340,600
실버	조지우	247,300
골드	임현우	181,000
VIP	오민준	127,800

4행

8.

```sql
WITH m AS (SELECT substr(order_date, 1, 7) AS 월, COUNT(*) AS n FROM orders GROUP BY 1)
SELECT 월, n AS 주문수, n - LAG(n) OVER (ORDER BY 월) AS 증감
FROM m ORDER BY 월;
```

월	주문수	증감
2026-01	56	*NULL*
2026-02	59	3
2026-03	54	-5
2026-04	47	-7
2026-05	53	6
2026-06	60	7
2026-07	57	-3
2026-08	47	-10
2026-09	67	20

9행

9.

```sql
WITH s AS (
  SELECT p.name, SUM(o.qty * p.price) AS 매출
  FROM orders o JOIN products p ON p.id = o.product_id
  WHERE o.status = '완료'
  GROUP BY p.name
)
SELECT name, 매출,
       ROUND(100.0 * 매출 / SUM(매출) OVER (), 1) AS 비중,
       RANK() OVER (ORDER BY 매출 DESC) AS 순위
FROM s
ORDER BY 순위
LIMIT 5;
```

name	매출	비중	순위
텀블러	1,175,000	17.2	1
원두 200g	1,170,000	17.1	2
머그컵	846,000	12.4	3
드립백 세트	696,000	10.2	4
샐러드	433,500	6.3	5

5행